# UTS

# PEMBELAJARAN MESIN




NAMA : Syalwa Melza Eka Fitri

NIM : 202531077

KELAS : H

DOSEN : Rizqia Cahyaningtyas, ST., M.Kom

NO.PC : 06

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Ardy

## INSTITUT TEKNOLOGI PLN

## TEKNIK INFORMATIKA

## 2026/2027

# DAFTAR ISI

DAFTAR ISI ..... 2
BAB I PENDAHULUAN ..... 3
1.1 Rumusan Masalah ..... 3
1.2 Tujuan Masalah ..... 3
1.3 Manfaat Masalah ..... 3
BAB II LANDASAN TEORI ..... 4
2.1 Machine Learning dan Klasifikasi ..... 4
2.2 EDA dan Preprocessing Data ..... 4
2.3 Decision Tree ..... 4
2.4 K-Nearest Neighbor (KNN) ..... 4
BAB III HASIL ..... 6
3.1 Import Library ..... 6
3.2 Load Dataset ..... 6
3.3 Exploratory Data Analysis (EDA) ..... 7
3.4 Preprocessing Data ..... 11
3.5 Pembagian Data Latih dan Data Uji ..... 12
3.6 Klasifikasi dengan Decision Tree ..... 13
3.7 Modelling KNN ..... 16
3.8 Evaluasi dan Perbandingan Model ..... 19
BAB IV PENUTUP ..... 22
DAFTAR PUSTAKA ..... 23

# BAB I
# PENDAHULUAN

## 1.1 Rumusan Masalah

Counter Strike 2 (CS2) adalah game tembak-tembakan tim, yaitu tim Counter-Terrorist (CT) melawan tim Terrorist (T). Di setiap ronde ada banyak kondisi yang berubah-ubah, misalnya sisa waktu, darah (health), armor, uang, dan apakah bom sudah ditanam. Dari data kondisi tersebut, muncul pertanyaan apakah pemenang ronde bisa ditebak menggunakan machine learning. Rumusan masalah pada UTS ini adalah:

a. Bagaimana cara melakukan EDA dan preprocessing pada dataset Counter Strike 2 supaya datanya siap dipakai untuk pemodelan?
b. Bagaimana penerapan algoritma Decision Tree dan K-Nearest Neighbor (KNN) untuk memprediksi pemenang ronde (CT atau T)?
c. Berapa akurasi yang didapat dari masing-masing algoritma, dan algoritma mana yang hasilnya lebih baik pada dataset ini?

## 1.2 Tujuan Masalah

a. Melakukan EDA untuk memahami isi dataset dan melakukan preprocessing (hapus duplikat dan mengubah data kategori menjadi angka).
b. Membuat model klasifikasi pemenang ronde menggunakan Decision Tree dan KNN.
c. Mengetahui dan membandingkan akurasi kedua model. Dari hasil praktikum, Decision Tree mendapat akurasi 85,95% dan KNN mendapat akurasi 89,00%.

## 1.3 Manfaat Masalah

a. Bagi saya sebagai mahasiswa, jadi lebih paham alur kerja machine learning mulai dari EDA, preprocessing, training, sampai evaluasi.
b. Mengetahui perbedaan cara kerja Decision Tree dan KNN, serta kenapa KNN perlu data yang sudah diskalakan.
c. Hasilnya bisa jadi gambaran sederhana faktor apa saja yang berpengaruh pada kemenangan ronde di game CS2.

# BAB II
# LANDASAN TEORI

## 2.1 Machine Learning dan Klasifikasi

Machine learning adalah cabang dari kecerdasan buatan di mana komputer belajar dari data untuk membuat prediksi. Salah satu jenisnya adalah supervised learning, yaitu model dilatih menggunakan data yang sudah punya label (jawaban). Klasifikasi termasuk supervised learning yang tujuannya menebak kelas dari suatu data. Pada project ini kelasnya ada dua, yaitu CT dan T, sehingga disebut klasifikasi biner.

## 2.2 EDA dan Preprocessing Data

Exploratory Data Analysis (EDA) adalah tahap awal untuk mengenal data, misalnya melihat jumlah data, tipe data, data kosong, data duplikat, sebaran kelas, dan hubungan (korelasi) antar fitur. Setelah itu dilakukan preprocessing, yaitu menyiapkan data supaya bisa dibaca model, contohnya menghapus data duplikat dan mengubah data teks menjadi angka (encoding). Untuk data kategori yang tidak punya urutan, seperti nama map, dipakai one-hot encoding (get_dummies), yaitu setiap kategori dibuat menjadi kolom sendiri yang berisi 0 atau 1. Cara ini dipakai supaya model tidak menganggap map yang satu "lebih besar" dari map yang lain.

Untuk algoritma yang memakai jarak seperti KNN, skala fitur juga perlu diperhatikan. Skala nilai fitur yang tidak seimbang bisa menghambat kinerja KNN, sehingga perlu dilakukan normalisasi (Allorerung dkk., 2024). Whendasmoro dan Joseph (2022) juga menunjukkan bahwa normalisasi Z-Score dapat menaikkan akurasi KNN dibandingkan sebelum data dinormalisasi.

## 2.3 Decision Tree

Decision Tree (pohon keputusan) adalah algoritma klasifikasi yang bentuknya seperti pohon, terdiri dari node (simpul), cabang, dan daun (leaf). Setiap node berisi pertanyaan tentang suatu fitur, misalnya "apakah t_helmets <= 1,5?", lalu data dibagi ke cabang kiri atau kanan sampai akhirnya sampai di daun yang berisi hasil kelas. Pada library scikit-learn, Decision Tree memakai algoritma CART dengan kriteria pemisahan Gini index sebagai bawaan (Harriz & Setiyowati, 2023). Semakin kecil nilai Gini, berarti data di node itu semakin "murni" atau semakin banyak dari satu kelas saja.

Salah satu parameter penting adalah max_depth, yaitu batas kedalaman pohon. Pohon yang terlalu dangkal bisa kurang belajar (underfitting), sedangkan pohon yang terlalu dalam bisa terlalu hafal data latih (overfitting). Overfitting bisa dilihat dari akurasi data latih yang sangat tinggi tetapi akurasi data ujinya jauh lebih rendah.

## 2.4 K-Nearest Neighbor (KNN)

K-Nearest Neighbor (KNN) adalah algoritma supervised yang menentukan kelas data baru berdasarkan kelas dari tetangga-tetangga terdekatnya. Jarak antar data biasanya dihitung dengan Euclidean distance (Minkowski dengan p = 2), dan nilai K menentukan berapa banyak tetangga yang ikut "voting" untuk menentukan kelas (Harriz & Setiyowati, 2023). Rumus Euclidean distance adalah:

$$d(x, y) = \sqrt{( \Sigma (x_i - y_i)^2 )}$$

Karena berbasis jarak, fitur yang nilainya besar (contohnya uang yang bisa sampai puluhan ribu) akan lebih "mendominasi" dibanding fitur yang nilainya kecil (contohnya jumlah helm 0 sampai 5). Itu sebabnya data perlu diskalakan dulu sebelum masuk ke KNN.

# BAB III
# HASIL

Pada bab ini dijelaskan hasil praktikum yang dikerjakan di Jupyter Notebook. Urutannya mengikuti alur: import library, EDA, preprocessing, modelling Decision Tree, modelling KNN, evaluasi, dan prediksi data baru.

## 3.1 Import Library

### Import library ¶

```
# 202531077_Syalwa Melza Eka Fitri
nim = "202531077_Syalwa Melza Eka Fitri"
print("NIM :", nim)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

sns.set(style="whitegrid")
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```

Cell ini mengimport library yang dipakai :

- import numpy as np digunakan untuk mengimpor NumPy dengan alias np, dipakai untuk perhitungan numerik.
- import pandas as pd digunakan untuk mengimpor Pandas dengan alias pd untuk membaca dan mengolah data dalam bentuk tabel (DataFrame).
- import matplotlib.pyplot as plt digunakan untuk mengimpor modul pyplot dari Matplotlib dengan alias plt untuk membuat dan menampilkan grafik.
- import seaborn as sns digunakan untuk mengimpor Seaborn dengan alias sns, pustaka grafik yang dibangun di atas Matplotlib dan penulisannya lebih ringkas.
- sns.set(style="whitegrid") digunakan untuk mengatur tema grafik menjadi latar putih dengan garis bantu (grid), agar grafik mudah dibaca.

## 3.2 Load Dataset

### Membaca dataset

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
df = pd.read_csv("03_counter_strike_2.csv")
df.head()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```

| | time_left | ct_score | t_score | map | bomb_planted | ct_health | t_health | ct_armor | t_armor | ct_money | t_money | ct_helmets | t_helmets | round_winner |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 175.00 | 0.0 | 0.0 | de_dust2 | False | 500.0 | 500.0 | 0.0 | 0.0 | 4000.0 | 4000.0 | 0.0 | 0.0 | CT |
| 1 | 156.03 | 0.0 | 0.0 | de_dust2 | False | 500.0 | 500.0 | 400.0 | 300.0 | 600.0 | 650.0 | 0.0 | 0.0 | CT |
| 2 | 96.03 | 0.0 | 0.0 | de_dust2 | False | 391.0 | 400.0 | 294.0 | 200.0 | 750.0 | 500.0 | 0.0 | 0.0 | CT |
| 3 | 76.03 | 0.0 | 0.0 | de_dust2 | False | 391.0 | 400.0 | 294.0 | 200.0 | 750.0 | 500.0 | 0.0 | 0.0 | CT |
| 4 | 174.97 | 1.0 | 0.0 | de_dust2 | False | 500.0 | 500.0 | 192.0 | 0.0 | 18350.0 | 10750.0 | 0.0 | 0.0 | CT |

**Membaca Dataset:** Kode ini membaca file 03_counter_strike_2.csv dan menampilkan lima baris pertamanya.

- df = pd.read_csv("03_counter_strike_2.csv") digunakan untuk membaca file CSV, lalu menyimpannya ke variabel df dalam bentuk DataFrame.
- df.head() digunakan untuk menampilkan lima baris pertama data agar isi dan bentuk datanya terlihat.

Dataset berisi kondisi saat ronde Counter Strike 2 berlangsung, yaitu sisa waktu (time_left), skor, map, status bom (bomb_planted), health, armor, uang, dan jumlah helm tiap tim. Kolom round_winner adalah target yang ingin ditebak, berisi CT atau T.

### 3.3 Exploratory Data Analysis (EDA)

Melihat informasi dataset (EDA)

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
df.info()
df.describe()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
<class 'pandas.DataFrame'>
RangeIndex: 2000 entries, 0 to 1999
Data columns (total 14 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   time_left     2000 non-null   float64
 1   ct_score      2000 non-null   float64
 2   t_score       2000 non-null   float64
 3   map           2000 non-null   str
 4   bomb_planted  2000 non-null   bool
 5   ct_health     2000 non-null   float64
 6   t_health      2000 non-null   float64
 7   ct_armor      2000 non-null   float64
 8   t_armor       2000 non-null   float64
 9   ct_money      2000 non-null   float64
 10  t_money       2000 non-null   float64
 11  ct_helmets    2000 non-null   float64
 12  t_helmets     2000 non-null   float64
 13  round_winner  2000 non-null   str
dtypes: bool(1), float64(11), str(2)
memory usage: 224.8 KB
```

| | time_left | ct_score | t_score | ct_health | t_health | ct_armor | t_armor | ct_money | t_money | ct_helmets | t_helmets |
|---|---|---|---|---|---|---|---|---|---|---|---|
| count | 2000.000000 | 2000.000000 | 2000.00000 | 2000.000000 | 2000.000000 | 2000.000000 | 2000.00000 | 2000.000000 | 2000.000000 | 2000.000000 | 2000.000000 |
| mean | 95.630205 | 6.840000 | 6.28850 | 406.170500 | 402.987500 | 306.749000 | 309.54450 | 7867.350000 | 12377.600000 | 1.821500 | 3.009500 |
| std | 53.734491 | 4.233599 | 4.19059 | 137.616258 | 137.699186 | 172.857221 | 168.68957 | 9017.491838 | 12209.581798 | 1.774884 | 1.960952 |
| min | 0.050000 | 0.000000 | 0.00000 | 2.000000 | 0.000000 | 0.000000 | 0.00000 | 0.000000 | 0.000000 | 0.000000 | 0.000000 |
| 25% | 54.910000 | 3.000000 | 3.00000 | 315.750000 | 328.750000 | 188.000000 | 190.00000 | 1200.000000 | 1700.000000 | 0.000000 | 1.000000 |
| 50% | 94.910000 | 7.000000 | 6.00000 | 500.000000 | 500.000000 | 300.000000 | 362.00000 | 4000.000000 | 9075.000000 | 1.000000 | 4.000000 |
| 75% | 156.425000 | 10.000000 | 10.00000 | 500.000000 | 500.000000 | 488.000000 | 467.00000 | 11762.500000 | 19250.000000 | 3.000000 | 5.000000 |
| max | 175.000000 | 15.000000 | 15.00000 | 500.000000 | 500.000000 | 500.000000 | 500.00000 | 58050.000000 | 67700.000000 | 5.000000 | 5.000000 |

**Informasi dan Statistik Deskriptif:** Kode ini menampilkan struktur dataset dan ringkasan statistik kolom numerik.

- df.info() digunakan untuk menampilkan jumlah baris, nama kolom, jumlah data yang terisi (Non-Null Count), dan tipe data tiap kolom.
- df.describe() digunakan untuk menghitung jumlah data (count), rata-rata (mean), simpangan baku (std), nilai minimum (min), kuartil 25%, 50% (median), 75%, dan nilai maksimum (max) tiap kolom numerik.

Dataset terdiri atas 2000 baris dan 14 kolom. Terdapat 11 kolom bertipe float64, 1 kolom bool (bomb_planted), dan 2 kolom teks (map dan round_winner), sehingga kolom teks dan bool perlu diubah menjadi angka. Skala tiap kolom juga berbeda jauh, misalnya ct_money maksimal 58.050 sedangkan ct_helmets maksimal hanya 5. Karena itu data perlu di-scaling sebelum dipakai KNN.

## Cek data kosong dan data duplikat

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
print("Data kosong:")
print(df.isnull().sum())
print("Data duplikat:", df.duplicated().sum())
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Data kosong:
time_left       0
ct_score        0
t_score         0
map             0
bomb_planted    0
ct_health       0
t_health        0
ct_armor        0
t_armor         0
ct_money        0
t_money         0
ct_helmets      0
t_helmets       0
round_winner    0
dtype: int64
Data duplikat: 39
```

**Cek Missing Values dan Data Duplikat:** Kode ini memeriksa data kosong dan menghitung baris duplikat.

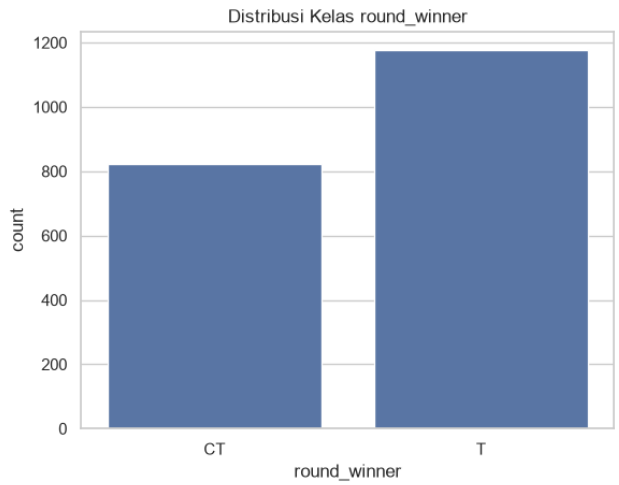
- df.isnull() menandai setiap sel yang kosong dengan True dan sel yang terisi dengan False. .sum() kemudian menjumlahkan nilai True pada tiap kolom, sehingga hasilnya adalah banyaknya data kosong per kolom.
- df.duplicated() menandai baris yang isinya sama persis dengan baris sebelumnya, lalu .sum() menjumlahkannya.

Semua kolom bernilai 0, sehingga dataset tidak memiliki missing values. Namun terdapat 39 baris duplikat yang akan dihapus pada tahap preprocessing, karena baris yang sama membuat model seolah-olah belajar dari data lebih banyak dari yang sebenarnya.

## Melihat sebaran kelas round_winner

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
sns.countplot(x="round_winner", data=df)
plt.title("Distribusi Kelas round_winner")
plt.show()
print(df["round_winner"].value_counts())
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```




```
round_winner
T     1176
CT     824
Name: count, dtype: int64
```

**Grafik Sebaran Kelas:** Kode ini membuat grafik batang jumlah pemenang ronde dan menghitung jumlah tiap kelas.

- sns.countplot(x="round_winner", data=df) digunakan untuk membuat grafik batang yang menghitung jumlah data di tiap kelas round_winner.
- plt.title("Distribusi Kelas round_winner") digunakan untuk memberi judul grafik.
- plt.figtext(0.99, 0.01, "NIM: " + nim, ha="right", color="gray") digunakan untuk menulis watermark NIM di pojok kanan bawah grafik. Angka 0.99 dan 0.01 adalah posisi (kanan, bawah) dalam skala 0 sampai 1.
- plt.show() digunakan untuk menampilkan grafik.
- df["round_winner"].value_counts() digunakan untuk menghitung jumlah data pada setiap kelas.

Tim T menang sebanyak 1176 kali (sekitar 59%) dan tim CT sebanyak 824 kali (sekitar 41%). Data sedikit tidak seimbang, tetapi masih wajar sehingga tidak perlu penyeimbangan data. Pembagian data nanti memakai stratify agar perbandingan ini tetap sama di data latih dan data uji.

## Boxplot hubungan fitur dengan pemenang ¶

```
#202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
plt.figure(figsize=(14, 4))

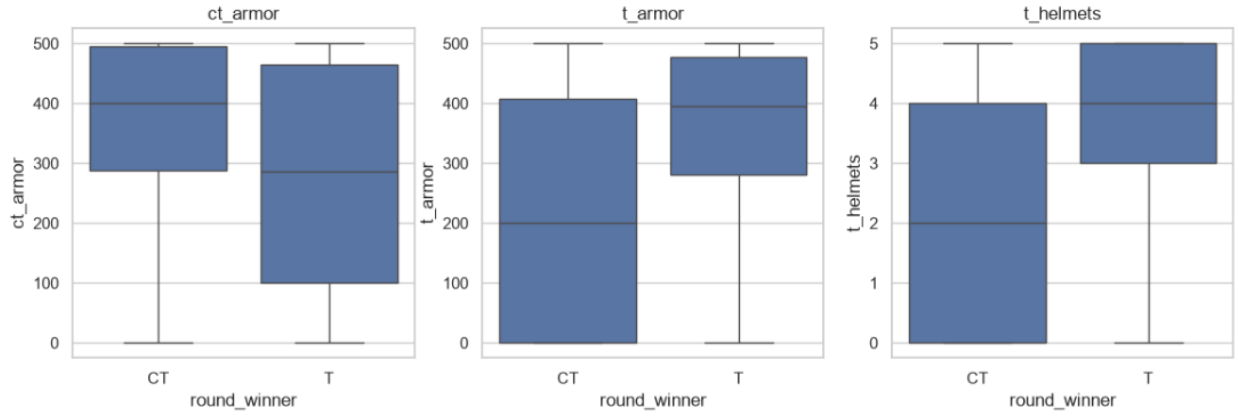
plt.subplot(1, 3, 1)
sns.boxplot(x="round_winner", y="ct_armor", data=df)
plt.title("ct_armor")

plt.subplot(1, 3, 2)
sns.boxplot(x="round_winner", y="t_armor", data=df)
plt.title("t_armor")

plt.subplot(1, 3, 3)
sns.boxplot(x="round_winner", y="t_helmets", data=df)
plt.title("t_helmets")

plt.show()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```




**Boxplot Fitur:** Kode ini membuat tiga boxplot untuk membandingkan nilai fitur saat CT menang dan saat T menang.

- plt.figure(figsize=(14, 4)) digunakan untuk membuat kanvas grafik berukuran 14 × 4 inci.
- plt.subplot(1, 3, 1) digunakan untuk membagi kanvas menjadi 1 baris 3 kolom dan memilih kolom pertama. Angka terakhir diganti 2 dan 3 untuk grafik berikutnya.
- sns.boxplot(x="round_winner", y="ct_armor", data=df) digunakan untuk membuat boxplot ct_armor yang dipisah berdasarkan pemenang. Garis di tengah kotak adalah median.

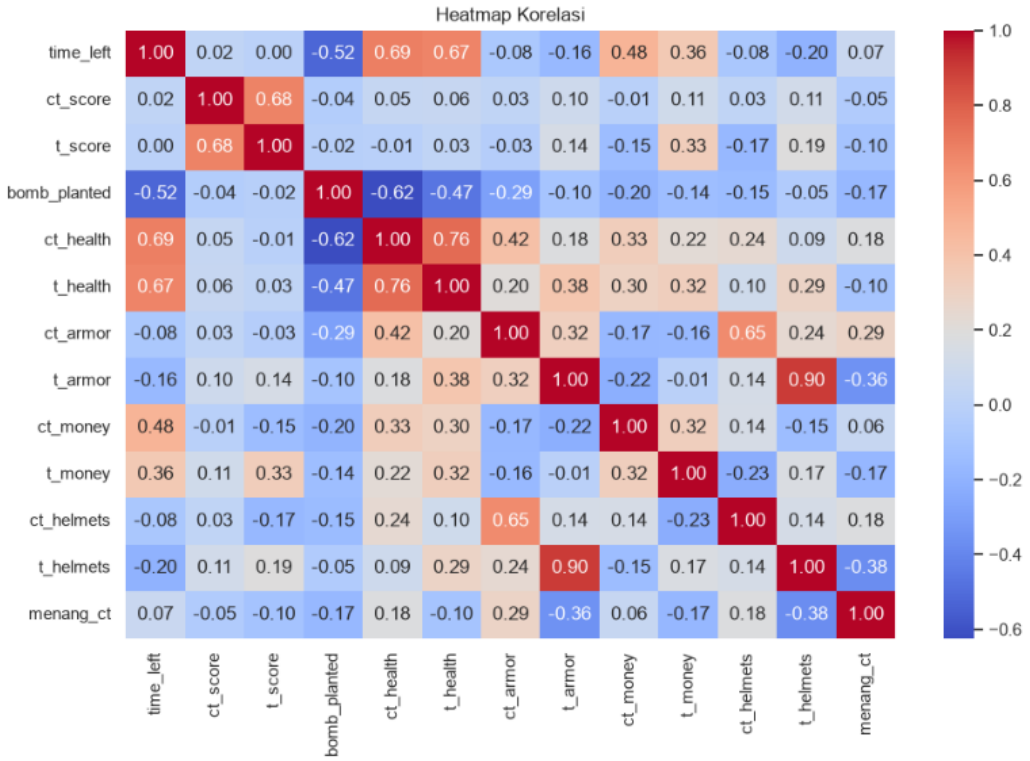
Median ct_armor saat CT menang adalah 400, sedangkan saat T menang hanya 286. Sebaliknya, median t_armor lebih tinggi saat T menang (394,5 dibanding 200), begitu juga t_helmets (4 dibanding 2). Artinya, tim yang perlengkapannya (armor dan helm) lebih lengkap cenderung memenangkan ronde.

## Heatmap korelasi

```
7]: # 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
df_korelasi = df.copy()
df_korelasi["bomb_planted"] = df_korelasi["bomb_planted"].astype(int)
df_korelasi["menang_ct"] = (df_korelasi["round_winner"] == "CT").astype(int)
angka = df_korelasi.select_dtypes(include=["int64", "float64"])

plt.figure(figsize=(11, 7))
sns.heatmap(angka.corr(), annot=True, fmt=".2f", cmap="coolwarm")
plt.title("Heatmap Korelasi")
plt.show()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```




**Heatmap Korelasi:** Kode ini membuat heatmap untuk melihat hubungan antarvariabel numerik, termasuk hubungannya dengan pemenang.

- df_korelasi = df.copy() digunakan untuk membuat salinan df, supaya perubahan untuk korelasi tidak mengubah data asli.
- df_korelasi["menang_ct"] = (df_korelasi["round_winner"] == "CT").astype(int) digunakan untuk membuat kolom bantu menang_ct yang bernilai 1 jika CT menang dan 0 jika T menang, karena korelasi hanya dapat dihitung untuk angka.
- df_korelasi.select_dtypes(include=["int64", "float64"]) digunakan untuk memilih hanya kolom bertipe angka.
- sns.heatmap(angka.corr(), annot=True, fmt=".2f", cmap="coolwarm") digunakan untuk membuat heatmap. corr() menghitung korelasi antarkolom, annot=True menampilkan angka di tiap kotak, fmt=".2f" membulatkan dua angka di belakang koma, dan cmap="coolwarm" memakai warna biru (korelasi negatif) sampai merah (korelasi positif).

Hubungan yang terlihat pada heatmap adalah sebagai berikut.

a. Variabel yang paling berkaitan dengan menang_ct adalah t_helmets (−0,38), t_armor (−0,36), dan ct_armor (0,29). Tanda negatif berarti makin banyak helm dan armor tim T, peluang CT menang makin kecil.
b. bomb_planted berkorelasi negatif dengan menang_ct (−0,17), artinya jika bom sudah ditanam, peluang T menang lebih besar.
c. t_armor dan t_helmets berkorelasi sangat kuat (0,90), sehingga keduanya membawa informasi yang mirip.
d. time_left berkorelasi negatif dengan bomb_planted (−0,52), karena bom biasanya ditanam saat waktu ronde sudah berjalan.

### 3.4 Preprocessing Data

Preprocessing: hapus duplikat dan ubah data jadi angka

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
df = df.drop_duplicates()
df["bomb_planted"] = df["bomb_planted"].astype(int)
df = pd.get_dummies(df, columns=["map"], dtype=int)

from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["round_winner"] = le.fit_transform(df["round_winner"])
for i, label in enumerate(le.classes_):
    print(f"{i} = {label}")

print("Jumlah data sekarang:", df.shape)
df.head()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
0 = CT
1 = T
Jumlah data sekarang: (1961, 20)
```

| | time_left | ct_score | t_score | bomb_planted | ct_health | t_health | ct_armor | t_armor | ct_money | t_money | ct_helmets | t_helmets | round_winner | map_de_dust2 | map_ |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 175.00 | 0.0 | 0.0 | 0 | 500.0 | 500.0 | 0.0 | 0.0 | 4000.0 | 4000.0 | 0.0 | 0.0 | 0 | 1 | |
| 1 | 156.03 | 0.0 | 0.0 | 0 | 500.0 | 500.0 | 400.0 | 300.0 | 600.0 | 650.0 | 0.0 | 0.0 | 0 | 1 | |
| 2 | 96.03 | 0.0 | 0.0 | 0 | 391.0 | 400.0 | 294.0 | 200.0 | 750.0 | 500.0 | 0.0 | 0.0 | 0 | 1 | |
| 3 | 76.03 | 0.0 | 0.0 | 0 | 391.0 | 400.0 | 294.0 | 200.0 | 750.0 | 500.0 | 0.0 | 0.0 | 0 | 1 | |
| 4 | 174.97 | 1.0 | 0.0 | 0 | 500.0 | 500.0 | 192.0 | 0.0 | 18350.0 | 10750.0 | 0.0 | 0.0 | 0 | 1 | |

**Pembersihan dan Encoding Data:** Kode ini menghapus data duplikat dan mengubah kolom teks serta boolean menjadi angka.

- df = df.drop_duplicates() digunakan untuk menghapus baris duplikat (satu baris dari tiap kelompok duplikat dipertahankan), lalu hasilnya disimpan kembali ke df.
- df["bomb_planted"].astype(int) digunakan untuk mengubah nilai True/False menjadi 1/0.
- pd.get_dummies(df, columns=["map"], dtype=int) digunakan untuk melakukan One-Hot Encoding pada kolom map. Setiap nama map menjadi kolom sendiri yang bernilai 1 jika baris tersebut memakai map itu dan 0 jika tidak. One-Hot Encoding dipilih karena nama map tidak memiliki urutan.
- le = LabelEncoder() digunakan untuk membuat objek Label Encoder, lalu le.fit_transform(df["round_winner"]) mengubah kelas round_winner menjadi angka. Kelas diurutkan sesuai abjad sehingga CT = 0 dan T = 1.
- for i, label in enumerate(le.classes_) digunakan untuk menampilkan arti setiap angka hasil encoding.

- print("Jumlah data sekarang:", df.shape) digunakan untuk menampilkan ukuran data setelah preprocessing.

Setelah 39 duplikat dihapus, data tersisa **1961 baris**. Jumlah kolom menjadi 20 karena satu kolom map diganti menjadi tujuh kolom (map_de_dust2, map_de_inferno, map_de_mirage, map_de_nuke, map_de_overpass, map_de_train, dan map_de_vertigo).

### 3.5 Pembagian Data Latih dan Data Uji

Memisahkan fitur (x) dan target (y)

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
x = df.drop("round_winner", axis=1)
y = df["round_winner"]
print("Bentuk x:", x.shape)
print("Bentuk y:", y.shape)
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Bentuk x: (1961, 19)
Bentuk y: (1961,)
```

**Menentukan Variabel x dan y:** Kode ini menentukan variabel input dan variabel target.

- x = df.drop("round_winner", axis=1) digunakan untuk mengambil semua kolom kecuali round_winner sebagai input. axis=1 berarti yang dihapus adalah kolom, bukan baris.
- y = df["round_winner"] digunakan untuk memilih kolom round_winner sebagai target, yaitu 0 (CT) atau 1 (T).
- x.shape dan y.shape digunakan untuk menampilkan ukuran x dan y.

  x berukuran (1961, 19), yaitu 1961 baris dengan 19 fitur, sedangkan y berukuran (1961,).

Membagi data latih dan data uji

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn.model_selection import train_test_split

x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=1, stratify=y)
print("x_train:", x_train.shape)
print("x_test :", x_test.shape)
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
x_train: (1470, 19)
x_test : (491, 19)
```

**Membagi Data Latih dan Data Uji:** Kode ini membagi data menjadi data latih dan data uji.

- x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.25, random_state=1, stratify=y) digunakan untuk membagi data. test_size=0.25 berarti 25% data menjadi data uji dan 75% data latih. random_state=1 mengunci pengacakan agar pembagian selalu sama setiap kode dijalankan. stratify=y menjaga perbandingan CT dan T di data latih dan data uji sama seperti data asli.

Dari 1961 data, sebanyak **1470 data** menjadi data latih dan **491 data** menjadi data uji. Data latih dan data uji yang sama dipakai untuk Decision Tree dan KNN, sehingga hasil kedua model bisa dibandingkan secara adil.

### 3.6 Klasifikasi dengan Decision Tree

Decision Tree - model pertama (pengaturan bawaan)

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

dt1 = DecisionTreeClassifier(random_state=1)
dt1.fit(x_train, y_train)

print("Kedalaman pohon:", dt1.get_depth())
print("Akurasi data latih:", accuracy_score(y_train, dt1.predict(x_train)))
print("Akurasi data uji  :", accuracy_score(y_test, dt1.predict(x_test)))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Kedalaman pohon: 16
Akurasi data latih: 0.9993197278911564
Akurasi data uji  : 0.8574338085539714
```

**Membuat dan Melatih Decision Tree:** Kode ini membuat model Decision Tree dengan pengaturan bawaan, lalu menghitung akurasinya di data latih dan data uji.

- from sklearn.tree import DecisionTreeClassifier digunakan untuk mengimpor kelas model Decision Tree, dan from sklearn.metrics import accuracy_score untuk mengimpor fungsi penghitung akurasi.
- dt1 = DecisionTreeClassifier(random_state=1) digunakan untuk membuat model Decision Tree dengan kriteria bawaan gini dan tanpa batas kedalaman. random_state=1 membuat hasilnya selalu sama.
- dt1.fit(x_train, y_train) digunakan untuk melatih model, yaitu membentuk pohon keputusan dari data latih.
- dt1.get_depth() digunakan untuk menampilkan kedalaman pohon yang terbentuk.
- accuracy_score(y_train, dt1.predict(x_train)) digunakan untuk menghitung akurasi pada data latih. predict() menebak kelas, lalu accuracy_score membandingkan tebakan dengan nilai asli. Cara yang sama dipakai untuk data uji.

Pohon yang terbentuk memiliki kedalaman 16. Akurasi data latih mencapai 99,93%, tetapi akurasi data uji hanya **85,74%**. Selisih yang jauh ini menunjukkan model terlalu menghafal data latih (**overfitting**), sehingga kedalaman pohon perlu dibatasi.

Decision Tree - mencoba beberapa max_depth

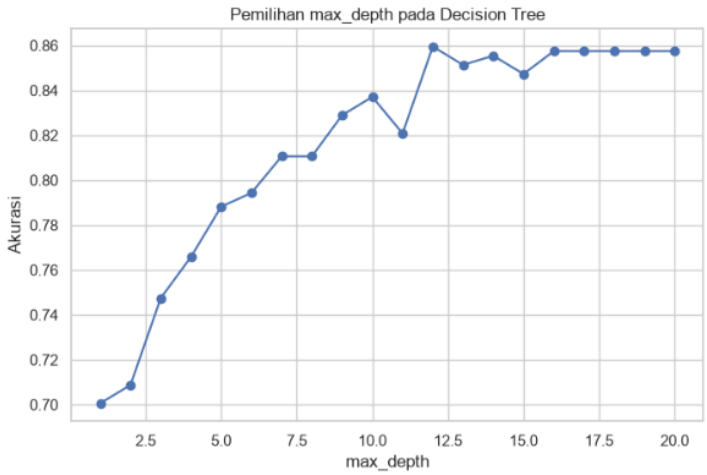
```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
depths = range(1, 21)
akurasi_dt = []

for d in depths:
    model = DecisionTreeClassifier(criterion="gini", max_depth=d, random_state=1)
    model.fit(x_train, y_train)
    akurasi_dt.append(accuracy_score(y_test, model.predict(x_test)))

plt.figure(figsize=(8, 5))
plt.plot(depths, akurasi_dt, marker="o")
plt.xlabel("max_depth")
plt.ylabel("Akurasi")
plt.title("Pemilihan max_depth pada Decision Tree")
plt.show()

best_depth = depths[akurasi_dt.index(max(akurasi_dt))]
print("max_depth terbaik:", best_depth)
print("akurasi:", max(akurasi_dt))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```




```
max_depth terbaik: 12
akurasi: 0.8594704684317719
```

**Pemilihan max_depth:** Kode ini mencoba max_depth dari 1 sampai 20 dan menampilkan akurasinya dalam grafik.

- depths = range(1, 21) digunakan untuk menentukan nilai max_depth yang dicoba, dan akurasi_dt = [] untuk menyimpan akurasi setiap percobaan.
- for d in depths: digunakan untuk mengulang pembuatan model. Setiap model dibuat dengan DecisionTreeClassifier(criterion="gini", max_depth=d, random_state=1), dilatih dengan fit(), lalu akurasinya di data uji disimpan dengan append().
- plt.plot(depths, akurasi_dt, marker="o") digunakan untuk membuat grafik garis akurasi terhadap max_depth. marker="o" memberi titik pada setiap nilai.
- best_depth = depths[akurasi_dt.index(max(akurasi_dt))] digunakan untuk mengambil max_depth yang akurasinya paling tinggi.

Akurasi naik dari 70,1% pada max_depth = 1 dan mencapai nilai tertinggi pada **max_depth = 12** dengan akurasi 85,95%. Setelah itu akurasi naik-turun dan stabil di 85,74%, sama seperti model tanpa batas. Jadi pohon yang lebih dalam dari 12 tidak menambah akurasi.

### Decision Tree - model akhir dan akurasi

```
13]: # 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn.metrics import classification_report

dt = DecisionTreeClassifier(criterion="gini", max_depth=best_depth, random_state=1)
dt.fit(x_train, y_train)
pred_dt = dt.predict(x_test)

akurasi_dt_final = accuracy_score(y_test, pred_dt)
print("Akurasi Decision Tree:", akurasi_dt_final)
print(classification_report(y_test, pred_dt, target_names=le.classes_))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Akurasi Decision Tree: 0.8594704684317719
              precision    recall  f1-score   support

          CT       0.83      0.83      0.83       202
           T       0.88      0.88      0.88       289

    accuracy                           0.86       491
   macro avg       0.85      0.86      0.86       491
weighted avg       0.86      0.86      0.86       491
```

**Evaluasi Decision Tree:** Kode ini membuat model Decision Tree akhir dengan max_depth terbaik dan menampilkan laporan evaluasinya.

- dt = DecisionTreeClassifier(criterion="gini", max_depth=best_depth, random_state=1) digunakan untuk membuat model akhir dengan max_depth = 12, lalu dt.fit(x_train, y_train) melatihnya.
- pred_dt = dt.predict(x_test) digunakan untuk memprediksi kelas data uji dan menyimpannya di pred_dt.
- classification_report(y_test, pred_dt, target_names=le.classes_) digunakan untuk menampilkan precision, recall, dan f1-score tiap kelas. target_names membuat kelas tampil sebagai CT dan T, bukan 0 dan 1.

Akurasi Decision Tree adalah **85,95%**. Kelas CT memiliki precision dan recall 0,83, sedangkan kelas T 0,88. Kelas T lebih mudah ditebak karena jumlah datanya lebih banyak.

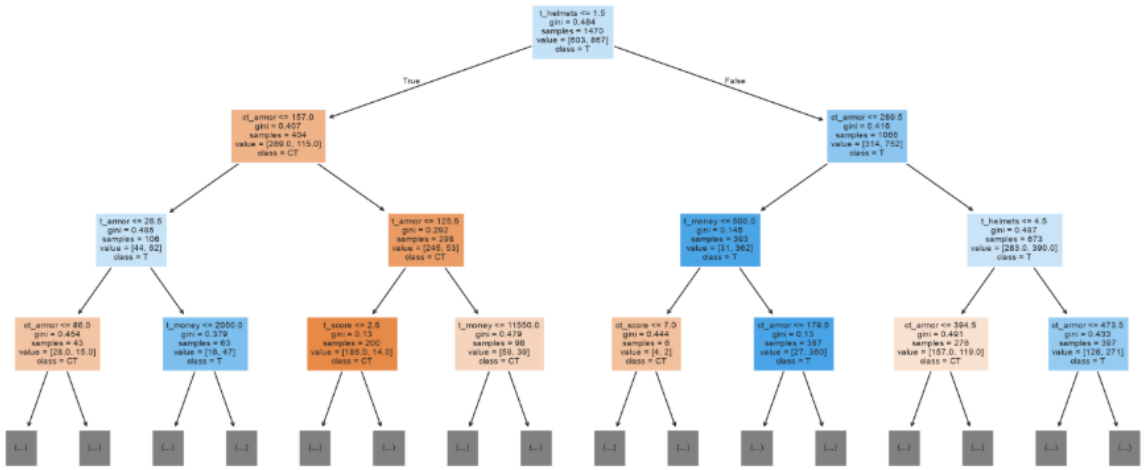
### Decision Tree - gambar pohon

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn import tree

plt.subplots(figsize=(18, 8))
tree.plot_tree(dt, max_depth=3, feature_names=x.columns, class_names=le.classes_, filled=True, fontsize=7)
plt.show()

penting = pd.Series(dt.feature_importances_, index=x.columns).sort_values(ascending=False)
print(penting.head(6))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```




```
t_helmets    0.191378
ct_armor     0.181863
ct_money     0.105795
t_money      0.089228
t_armor      0.088854
ct_score     0.063157
dtype: float64
```

**Visualisasi Pohon Keputusan:** Kode ini menggambar pohon keputusan dan menampilkan fitur yang paling berpengaruh.

- plt.subplots(figsize=(18, 8)) digunakan untuk membuat kanvas berukuran 18 × 8 inci karena pohonnya lebar.
- tree.plot_tree(dt, max_depth=3, feature_names=x.columns, class_names=le.classes_, filled=True, fontsize=7) digunakan untuk menggambar pohon. max_depth=3 hanya membatasi gambar supaya terbaca, model aslinya tetap 12 level. filled=True memberi warna sesuai kelas mayoritas (oranye = CT, biru = T).

- pd.Series(dt.feature_importances_, index=x.columns).sort_values(ascending=False) digunakan untuk mengambil nilai kepentingan tiap fitur dan mengurutkannya dari yang terbesar. head(6) menampilkan enam fitur teratas.

Node paling atas (root) memakai pertanyaan t_helmets <= 1,5 dengan 1470 data latih. Setiap kotak berisi kondisi pemisah, nilai gini, jumlah data (samples), jumlah tiap kelas (value), dan kelas mayoritas. Fitur paling penting adalah t_helmets (0,191), ct_armor (0,182), dan ct_money (0,106). Hasil ini sesuai dengan heatmap, di mana helm dan armor paling berkaitan dengan pemenang.

### 3.7 Modelling KNN

KNN - scaling data

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
x_train_scaled = scaler.fit_transform(x_train)
x_test_scaled = scaler.transform(x_test)

print("Rata-rata kolom ct_money sebelum scaling:", round(x_train["ct_money"].mean(), 1))
print("Rata-rata kolom ct_money sesudah scaling:", round(x_train_scaled[:, list(x.columns).index("ct_money")].mean(), 4))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Rata-rata kolom ct_money sebelum scaling: 7704.9
Rata-rata kolom ct_money sesudah scaling: 0.0
```

**Standarisasi Data:** Kode ini menyamakan skala semua fitur sebelum dipakai KNN.

- scaler = StandardScaler() digunakan untuk membuat objek standarisasi yang mengubah setiap kolom menjadi rata-rata 0 dan simpangan baku 1 dengan rumus z = (x − rata-rata) / simpangan baku.
- x_train_scaled = scaler.fit_transform(x_train) digunakan untuk menghitung rata-rata dan simpangan baku dari data latih (fit), lalu mengubah data latih (transform).
- x_test_scaled = scaler.transform(x_test) digunakan untuk mengubah data uji memakai rata-rata dan simpangan baku dari data latih, supaya informasi data uji tidak ikut bocor ke proses pelatihan.
- list(x.columns).index("ct_money") digunakan untuk mencari posisi kolom ct_money, karena hasil scaling berupa array yang tidak memiliki nama kolom.

Rata-rata ct_money yang awalnya 7704,9 menjadi 0,0 setelah scaling. Tanpa langkah ini, kolom uang yang bernilai ribuan akan mendominasi perhitungan jarak KNN, sedangkan kolom helm yang hanya 0 sampai 5 hampir tidak berpengaruh.

KNN - model pertama dengan k = 7

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn.neighbors import KNeighborsClassifier

knn1 = KNeighborsClassifier(n_neighbors=7)
knn1.fit(x_train_scaled, y_train)
pred_knn1 = knn1.predict(x_test_scaled)
print("Akurasi KNN (k=7):", accuracy_score(y_test, pred_knn1))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Akurasi KNN (k=7): 0.8228105906313645
```

**Membuat dan Melatih KNN:** Kode ini membuat model KNN dengan 7 tetangga terdekat dan menghitung akurasinya.

- from sklearn.neighbors import KNeighborsClassifier digunakan untuk mengimpor kelas model KNN.
- knn1 = KNeighborsClassifier(n_neighbors=7) digunakan untuk membuat model KNN yang melihat 7 tetangga terdekat. Jarak yang dipakai secara bawaan adalah Euclidean.
- knn1.fit(x_train_scaled, y_train) digunakan untuk melatih model. Pada KNN, fit hanya menyimpan data latih, perhitungan jarak baru dilakukan saat predict().

  Akurasi KNN dengan K = 7 adalah **82,28%**, masih lebih rendah dari Decision Tree.

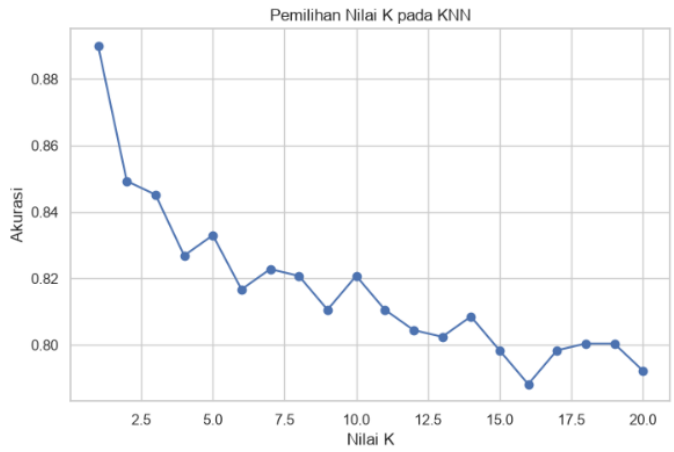
**KNN - mencari nilai k terbaik**

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
k_values = range(1, 21)
akurasi_knn = []

for k in k_values:
    model = KNeighborsClassifier(n_neighbors=k)
    model.fit(x_train_scaled, y_train)
    akurasi_knn.append(accuracy_score(y_test, model.predict(x_test_scaled)))

plt.figure(figsize=(8, 5))
plt.plot(k_values, akurasi_knn, marker="o")
plt.xlabel("Nilai K")
plt.ylabel("Akurasi")
plt.title("Pemilihan Nilai K pada KNN")
plt.grid(True)
plt.show()

best_k = k_values[akurasi_knn.index(max(akurasi_knn))]
print("k terbaik:", best_k)
print("akurasi:", max(akurasi_knn))
```




```
k terbaik: 1
akurasi: 0.890020366598778
```

**Pemilihan Nilai K:** Kode ini mencoba nilai K dari 1 sampai 20 dan menampilkan akurasinya dalam grafik.

- k_values = range(1, 21) digunakan untuk menentukan nilai K yang dicoba, dan akurasi_knn = [] untuk menyimpan hasilnya.
- for k in k_values: digunakan untuk membuat model KNeighborsClassifier(n_neighbors=k) di setiap perulangan, melatihnya, lalu menyimpan akurasinya di data uji.
- plt.grid(True) digunakan untuk menampilkan garis bantu pada grafik.
- best_k = k_values[akurasi_knn.index(max(akurasi_knn))] digunakan untuk mengambil nilai K dengan akurasi tertinggi.

Akurasi tertinggi ada pada **K = 1** yaitu 89,00%. Semakin besar K, akurasi cenderung turun sampai sekitar 79% pada K = 20. Menurut saya hal ini terjadi karena dataset berisi beberapa catatan

dari ronde yang sama, misalnya baris 0 sampai 3 pada df.head() sama-sama ronde awal di de_dust2 dengan skor 0-0. Akibatnya banyak data yang sangat mirip, sehingga satu tetangga terdekat sudah cukup. Kekurangannya, K = 1 lebih mudah terpengaruh data yang aneh (noise).

### KNN - model akhir dan akurasi

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
knn = KNeighborsClassifier(n_neighbors=best_k)
knn.fit(x_train_scaled, y_train)
pred_knn = knn.predict(x_test_scaled)

akurasi_knn_final = accuracy_score(y_test, pred_knn)
print("Akurasi KNN:", akurasi_knn_final)
print(classification_report(y_test, pred_knn, target_names=le.classes_))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Akurasi KNN: 0.890020366598778
              precision    recall  f1-score   support

          CT       0.89      0.84      0.86       202
           T       0.89      0.93      0.91       289

    accuracy                           0.89       491
   macro avg       0.89      0.88      0.89       491
weighted avg       0.89      0.89      0.89       491
```

**Evaluasi KNN:** Kode ini membuat model KNN akhir dengan K terbaik dan menampilkan laporan evaluasinya.

- knn = KNeighborsClassifier(n_neighbors=best_k) digunakan untuk membuat model akhir dengan K = 1, lalu knn.fit(x_train_scaled, y_train) melatihnya.
- pred_knn = knn.predict(x_test_scaled) digunakan untuk memprediksi kelas data uji yang sudah di-scaling.
- classification_report(y_test, pred_knn, target_names=le.classes_) digunakan untuk menampilkan precision, recall, dan f1-score tiap kelas.

Akurasi KNN adalah **89,00%**. Precision kelas CT dan T sama-sama 0,89. Recall kelas CT 0,84 dan kelas T 0,93, artinya masih ada beberapa ronde yang dimenangkan CT tetapi ditebak T.

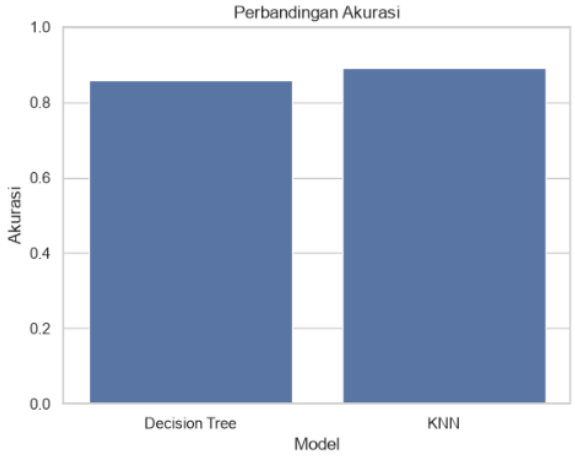
## 3.8 Evaluasi dan Perbandingan Model

### Perbandingan Decision Tree dan KNN ¶

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
hasil = pd.DataFrame({
    "Model": ["Decision Tree", "KNN"],
    "Akurasi": [akurasi_dt_final, akurasi_knn_final]
})
print(hasil)

sns.barplot(x="Model", y="Akurasi", data=hasil)
plt.ylim(0, 1)
plt.title("Perbandingan Akurasi")
plt.show()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
           Model  Akurasi
0  Decision Tree  0.85947
1            KNN  0.89002
```




**Perbandingan Akurasi:** Kode ini membandingkan akurasi kedua model dalam tabel dan grafik batang.

- hasil = pd.DataFrame({...}) digunakan untuk membuat tabel berisi nama model dan akurasinya.
- sns.barplot(x="Model", y="Akurasi", data=hasil) digunakan untuk membuat grafik batang perbandingan akurasi.
- plt.ylim(0, 1) digunakan untuk membuat sumbu y dari 0 sampai 1 agar tinggi batang dapat dibandingkan dengan adil.

Hasil perbandingan kedua model ditunjukkan pada tabel berikut.

| Model | Parameter Terbaik | Akurasi |
|---|---|---|
| Decision Tree | max_depth = 12 | 85,95% |
| KNN | K = 1 | 89,00% |

KNN lebih unggul sekitar 3% dibanding Decision Tree pada dataset ini.

## Confusion matrix

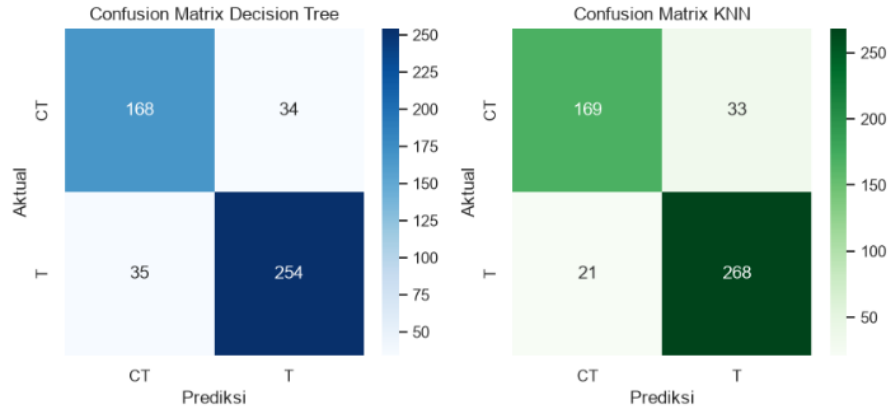
```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
from sklearn.metrics import confusion_matrix

fig, ax = plt.subplots(1, 2, figsize=(10, 4))
sns.heatmap(confusion_matrix(y_test, pred_dt), annot=True, fmt="d", cmap="Blues",
            xticklabels=le.classes_, yticklabels=le.classes_, ax=ax[0])
ax[0].set_title("Confusion Matrix Decision Tree")
ax[0].set_xlabel("Prediksi")
ax[0].set_ylabel("Aktual")

sns.heatmap(confusion_matrix(y_test, pred_knn), annot=True, fmt="d", cmap="Greens",
            xticklabels=le.classes_, yticklabels=le.classes_, ax=ax[1])
ax[1].set_title("Confusion Matrix KNN")
ax[1].set_xlabel("Prediksi")
ax[1].set_ylabel("Aktual")

plt.show()
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
```




**Visualisasi Confusion Matrix:** Kode ini menggambar confusion matrix kedua model secara berdampingan.

- fig, ax = plt.subplots(1, 2, figsize=(10, 4)) digunakan untuk membuat dua grafik berdampingan. ax[0] dipakai untuk Decision Tree dan ax[1] untuk KNN.
- sns.heatmap(confusion_matrix(y_test, pred_dt), annot=True, fmt="d", cmap="Blues", ...) digunakan untuk menggambar confusion matrix. annot=True menampilkan angka di tiap kotak, fmt="d" menampilkan angka sebagai bilangan bulat, dan cmap mengatur warna (biru untuk Decision Tree, hijau untuk KNN).
- set_xlabel("Prediksi") dan set_ylabel("Aktual") digunakan untuk memberi label sumbu x (kelas prediksi) dan sumbu y (kelas asli).

  Confusion matrix pada data uji (491 ronde) ditunjukkan pada tabel berikut.

| | **Decision Tree** | **KNN** |
|---|---|---|
| Asli CT, prediksi CT (benar) | 168 | 169 |
| Asli CT, prediksi T (salah) | 34 | 33 |
| Asli T, prediksi CT (salah) | 35 | 21 |
| Asli T, prediksi T (benar) | 254 | 268 |

Cara membaca hasilnya adalah sebagai berikut.

1. Decision Tree menebak benar 168 + 254 = 422 dari 491 data, sehingga akurasinya 422/491 = 0,8595.
2. KNN menebak benar 169 + 268 = 437 dari 491 data, sehingga akurasinya 437/491 = 0,8900.
3. Perbedaan terbesar ada pada ronde yang dimenangkan T tetapi ditebak CT. Decision Tree salah 35 kali, sedangkan KNN hanya 21 kali.

### 3.9 Prediksi Data Baru

Prediksi data baru

```
# 202531077_Syalwa Melza Eka Fitri
print("NIM :", nim)
#contoh kondisi: map de_mirage, sisa waktu 40 detik, bom sudah ditanam, tim T lebih unggul
data_baru = pd.DataFrame(0, index=[0], columns=x.columns)
data_baru["time_left"] = 40
data_baru["ct_score"] = 5
data_baru["t_score"] = 6
data_baru["bomb_planted"] = 1
data_baru["ct_health"] = 200
data_baru["t_health"] = 400
data_baru["ct_armor"] = 150
data_baru["t_armor"] = 300
data_baru["ct_money"] = 3000
data_baru["t_money"] = 5000
data_baru["ct_helmets"] = 2
data_baru["t_helmets"] = 4
data_baru["map_de_mirage"] = 1

print("Prediksi Decision Tree:", le.inverse_transform(dt.predict(data_baru)))
print("Prediksi KNN          :", le.inverse_transform(knn.predict(scaler.transform(data_baru))))
```

```
NIM : 202531077_Syalwa Melza Eka Fitri
Prediksi Decision Tree: ['T']
Prediksi KNN          : ['T']
```

**Prediksi Data Baru:** Kode ini memakai kedua model untuk menebak pemenang dari satu kondisi ronde baru.

- data_baru = pd.DataFrame(0, index=[0], columns=x.columns) digunakan untuk membuat satu baris data baru dengan 19 kolom yang sama seperti x, semuanya diisi 0 terlebih dahulu.
- data_baru["time_left"] = 40 dan baris-baris berikutnya digunakan untuk mengisi kondisi ronde. data_baru["map_de_mirage"] = 1 karena map yang dipakai adalah de_mirage, sedangkan kolom map lainnya tetap 0.
- dt.predict(data_baru) digunakan untuk menebak dengan Decision Tree. Data tidak perlu di-scaling karena Decision Tree tidak memakai jarak.
- knn.predict(scaler.transform(data_baru)) digunakan untuk menebak dengan KNN. Data baru harus di-scaling dulu dengan scaler yang sama seperti data latih.
- le.inverse_transform() digunakan untuk mengubah hasil prediksi berupa angka (0/1) kembali menjadi teks CT/T.

Kedua model sama-sama menebak **T** sebagai pemenang. Jalur yang dilewati data baru pada pohon Decision Tree ditunjukkan pada tabel berikut.

| Tahap | Pertanyaan di Node | Nilai Data Baru | Arah |
|---|---|---|---|
| 1 | t_helmets <= 1,5 | 4 | Tidak → kanan |
| 2 | ct_armor <= 289,5 | 150 | Ya → kiri |
| 3 | t_money <= 500 | 5000 | Tidak → kanan |
| 4 | ct_armor <= 179,5 | 150 | Ya → kiri |
| 5 | t_armor <= 157 | 300 | Tidak → kanan |
| 6 | time_left <= 174,99 | 40 | Ya → kiri |
| Daun | 198 data latih, semuanya T | - | Prediksi T |

Data baru melewati enam pertanyaan sebelum sampai di daun. Semua data latih di daun tersebut kelasnya T, sehingga Decision Tree menebak T. Hasil ini masuk akal karena kondisi tim T sedang lebih unggul, yaitu bom sudah ditanam serta helm, armor, dan uang tim T lebih banyak.

# BAB IV
# PENUTUP

Berdasarkan landasan teori dan hasil praktikum, dapat disimpulkan bahwa:

1. Dataset Counter Strike 2 berisi 2000 data dan 14 kolom tanpa data kosong. Ada 39 data duplikat yang dihapus sehingga tersisa 1961 data. Kolom map diubah dengan get_dummies, bomb_planted menjadi 1/0, dan round_winner dengan LabelEncoder.
2. Dari EDA, tim T lebih sering menang (sekitar 59%). Fitur yang paling berhubungan dengan pemenang adalah t_helmets, t_armor, dan ct_armor.
3. Decision Tree bawaan mengalami overfitting (akurasi latih 99,93%, uji 85,74%). Setelah max_depth dibatasi menjadi 12, akurasinya menjadi 85,95%. Fitur paling penting adalah t_helmets dan ct_armor.
4. KNN perlu data yang diskalakan dulu dengan StandardScaler karena berbasis jarak. Dengan K = 1, KNN mendapat akurasi 89,00%.
5. KNN lebih unggul sekitar 3% dibanding Decision Tree pada dataset ini, dan jumlah salah tebaknya juga lebih sedikit (54 dibanding 69). Hasil ini sejalan dengan penelitian Sinap (2023) pada data CS:GO yang juga mendapatkan akurasi KNN sedikit lebih tinggi dari Decision Tree.

Saran untuk pengembangan berikutnya, bisa dicoba cross validation supaya akurasinya lebih bisa dipercaya karena pembagian data di praktikum ini hanya dilakukan satu kali. Selain itu, nilai K = 1 sebaiknya dicek lagi dengan data lain karena K yang kecil lebih mudah terpengaruh data yang aneh.

## DAFTAR PUSTAKA

Allorerung, P. P., Erna, A., Bagussahrir, M., & Alam, S. (2024). Analisis performa normalisasi data untuk klasifikasi K-Nearest Neighbor pada dataset penyakit. *JISKA (Jurnal Informatika Sunan Kalijaga), 9*(3), 178–191. https://doi.org/10.14421/jiska.2024.9.3.178-191

Harriz, M. A., & Setiyowati, H. (2023). Komparasi algoritma Decision Tree dan KNN dalam mengklasifikasi daerah berdasarkan produksi listrik. *JIKO (Jurnal Informatika dan Komputer), 7*(2), 167–171. https://doi.org/10.26798/jiko.v7i2.787

Kasmara, B. N., Handayani, E. T. E., & Nathasia, N. D. (2024). Perbandingan kinerja algoritma K-Nearest Neighbors (K-NN) dan Decision Tree dalam deteksi paket malis pada jaringan. *STRING (Satuan Tulisan Riset dan Inovasi Teknologi), 8*(3), 320–329. https://journal.lppmunindra.ac.id/index.php/STRING/article/download/22362/6520

Lile, F. X. K., & Suharjo, I. (2024). Optimalisasi perbandingan algoritma K-Nearest Neighbors dan Decision Tree untuk prediksi kemenangan di MPL Season 13 Mobile Legend. *JATI (Jurnal Mahasiswa Teknik Informatika), 8*(4), 7553–7560. https://ejournal.itn.ac.id/jati/article/download/10343/5973

Sinap, V. (2023). Prediction of Counter-Strike: Global Offensive round results with machine learning techniques. *Journal of Intelligent Systems: Theory and Applications, 6*(2), 119–129. https://doi.org/10.38016/jista.1235031

Whendasmoro, R. G., & Joseph, J. (2022). Analisis penerapan normalisasi data dengan menggunakan Z-Score pada kinerja algoritma K-NN. *JURIKOM (Jurnal Riset Komputer), 9*(4), 872. https://doi.org/10.30865/jurikom.v9i4.4526